ARTIFICIAL BEE COLONY (ABC): ORIGEM, FUNCIONAMENTO E APLICAÇÃO EM OTIMIZAÇÃO NUMÉRICA

Matheus Beuren Carvalho
Disciplina de Pesquisa e Ordenação, professora Glória Patricia

Resumo: Este trabalho apresenta a metaheurística Artificial Bee Colony (ABC), proposta por Dervis Karaboga em 2005 a partir do comportamento de busca de alimento das abelhas, abordando como o algoritmo foi criado, quais são seus objetivos, como ele funciona, em que problemas e áreas é utilizado. Para mostrar a prática, o ABC foi implementado em Python e testado na minimização de x^2, da função Sphere e da função Rastrigin com duas variáveis, variando o limite de abandono das fontes de alimento e o número de fontes em 30 execuções independentes. Os resultados mostraram que um limite muito baixo atrapalha a convergência, pois a abelha exploradora entra quase todo ciclo e as fontes não chegam a ser refinadas, e que poucas fontes de alimento diminuem a taxa de acerto do mínimo global da Rastrigin. Conclui-se que o ABC é um método simples para otimização numérica sem derivadas, porém sensível aos seus parâmetros.

Palavras-chave: Artificial Bee Colony. Metaheurística. Otimização numérica. Inteligência de enxame.

Abstract: This paper presents the Artificial Bee Colony (ABC) metaheuristic, proposed by Dervis Karaboga in 2005 and inspired by the foraging behavior of honey bees. It covers how the algorithm was created, its main goals, how it works, and the problems and research areas where it is applied. As a practical example, ABC was implemented in Python and tested on the minimization of x^2, of the Sphere function and of the two-variable Rastrigin function, varying the abandonment limit and the number of food sources over 30 independent runs. Results showed that a very low limit hurts convergence, since the scout bee acts in almost every cycle and the food sources are never refined, and that few food sources lower the rate at which the global minimum of Rastrigin is found. ABC proved to be a simple derivative-free method for numerical optimization, although sensitive to its parameters.

Keywords: Artificial Bee Colony. Metaheuristic. Numerical optimization. Swarm intelligence.

1 INTRODUÇÃO

Muitos problemas de otimização têm um espaço de busca tão grande que testar todas as soluções não é viável, como no caso de um entregador que precisa passar por 20 paradas, onde existem cerca de 60 quadrilhões de rotas possíveis e, mesmo conferindo um bilhão delas por segundo, levaria quase dois anos para olhar todas. Nisso entram as metaheurísticas, que são estratégias gerais de busca que combinam sorteio com um critério de qualidade, explorando lugares novos e aproveitando os que já deram certo, assim não garantem a melhor solução mas costumam achar uma muito boa em pouco tempo.

O Artificial Bee Colony (ABC) é uma dessas metaheurísticas, inspirada na forma como uma colmeia procura néctar e proposta por Karaboga (2005). Ele foi criado para

otimização numérica, ou seja, encontrar os valores das variáveis de uma função que deixam o resultado o menor (ou o maior) possível.

Ao longo deste trabalho são apresentados como o ABC foi criado, quais são seus objetivos, como ele funciona, seu uso principal, as áreas de pesquisa e os problemas em que aparece, e um exemplo prático em que o algoritmo foi implementado em Python e testado em funções de teste, com os parâmetros variando.

2 ORIGEM E OBJETIVOS DO ALGORITMO

O ABC foi proposto por Dervis Karaboga no relatório técnico TR06 da Universidade Erciyes, na Turquia, em 2005, com o título de uma ideia baseada no enxame de abelhas para otimização numérica (KARABOGA, 2005). A versão publicada em periódico veio dois anos depois, em Karaboga e Basturk (2007), no Journal of Global Optimization, onde o algoritmo é descrito como um otimizador baseado no comportamento inteligente de um enxame de abelhas e comparado com um algoritmo genético (GA), com a otimização por enxame de partículas (PSO) e com um algoritmo evolutivo inspirado em partículas (PS-EA), em problemas numéricos de várias variáveis, e os autores relataram que o ABC teve desempenho superior aos demais. Já em Karaboga e Basturk (2008) a comparação foi com evolução diferencial, PSO e um algoritmo evolutivo, e o resultado foi mais modesto: o desempenho do ABC foi comparável ao dos outros, e ele se mostrou aplicável a problemas de engenharia de alta dimensão.

A inspiração está no que acontece numa colmeia real, onde as abelhas se dividem em funções: algumas ficam trabalhando numa fonte de alimento que já conhecem, outras esperam na colmeia e escolhem para qual fonte ir de acordo com o que as colegas trazem, e outras saem procurando fontes novas sem pista nenhuma. Essa divisão de trabalho sem um controle central é o que se chama de inteligência de enxame, área que estuda grupos de agentes simples que se organizam sozinhos (KARABOGA; BASTURK, 2007), e o PSO citado acima também faz parte dela.

Os objetivos do ABC são encontrar o valor que minimiza ou maximiza uma função, como o menor erro ou o menor custo, de forma simples, com poucos parâmetros de controle, que são o tamanho da colônia, o limite de abandono e o número máximo de ciclos (KARABOGA; BASTURK, 2008), sem precisar de derivadas da função e equilibrando a busca perto das soluções boas com a exploração de regiões novas.

3 FUNCIONAMENTO

No ABC cada fonte de alimento é uma solução possível do problema, ou seja, um conjunto de valores para as variáveis, e a quantidade de néctar é a qualidade dessa solução. Na minimização de uma função f, quanto menor o valor de f, mais néctar a fonte tem, e neste trabalho a qualidade foi calculada como 1/(1 + f), que é um valor que cresce quando f cai e é sempre positivo para as funções usadas. A colônia é dividida em duas metades, uma de abelhas empregadas e outra de observadoras, onde o número de empregadas é igual ao número de fontes, e ainda existe a abelha exploradora, geralmente uma por ciclo (KARABOGA; BASTURK, 2007).

A empregada fica na sua fonte e testa um ponto novo perto dela, escolhendo uma fonte vizinha k qualquer e uma variável j qualquer, e alterando só essa variável conforme a Equação 1, onde φ é um número sorteado entre -1 e 1.

$$v_{ij} = x_{ij} + \varphi(x_{ij} - x_{kj}) \quad (1)$$

O ponto novo v só substitui a fonte x se tiver mais néctar, e a comparação é feita com a própria fonte da abelha e nunca com a vizinha, que serve apenas para definir a direção e o tamanho do passo. Se o ponto novo é pior, a fonte continua onde estava e um contador de falhas daquela fonte aumenta em um, e se é melhor o contador volta a zero. É uma ideia parecida com a diferença entre soluções usada na evolução diferencial, só que aqui a vizinha é sorteada a cada visita.

Terminadas as empregadas, cada fonte recebe uma chance de ser escolhida proporcional à sua qualidade, como numa roleta onde as fatias maiores são das fontes com mais néctar (Equação 2), e as observadoras sorteiam fontes por essa roleta e repetem a mesma visita da empregada. Assim a busca se concentra onde as soluções são melhores, sem abandonar de vez as piores, que ainda têm uma fatia pequena.

$$p_i = fit_i / \Sigma_j fit_j \quad (2)$$

Por fim o melhor ponto encontrado é guardado e a exploradora age: se a fonte com mais falhas passou do limite, ela é abandonada e a abelha sorteia um ponto qualquer dentro do domínio, que é aceito mesmo se for pior. O limite é o parâmetro que regula o equilíbrio entre aproveitar e explorar, porque com um limite baixo a colônia troca de fonte cedo e com um limite alto insiste nas que já tem. O ciclo inteiro, empregadas, roleta, observadoras, registro do melhor e exploradora, se repete até o número máximo de ciclos.

4 USOS, ÁREAS DE PESQUISA E PROBLEMAS RESOLVIDOS

O principal uso do ABC é a otimização numérica, que é achar valores que minimizam um erro ou um custo quando a função não precisa ser derivável, e foi nesse tipo de problema que ele nasceu e foi validado, em funções de várias variáveis comparadas com outros algoritmos (KARABOGA; BASTURK, 2007; 2008). Funções de teste como a Sphere, que tem um vale só, e a Rastrigin, que tem dezenas de vales falsos, são usadas justamente porque deixam claro quando um método fica preso num mínimo local.

Uma aplicação bem direta é o treinamento de redes neurais. Treinar uma rede é um problema de otimização, onde se procura o conjunto de pesos que deixa o erro o menor possível, e então cada fonte de alimento passa a ser um conjunto de pesos inteiro. Karaboga, Akay e Öztürk (2007) usaram o ABC para treinar redes neurais de múltiplas camadas e compararam com o algoritmo genético e com o *backpropagation*, motivados pelo fato de que os algoritmos tradicionais de treino têm desvantagens como ficar preso em mínimos locais e a complexidade computacional.

Em termos de áreas de pesquisa, a revisão de Karaboga et al. (2014) trata o ABC como o algoritmo de inteligência de enxame inspirado em abelhas mais estudado e aplicado a problemas reais até então, reunindo suas variantes e aplicações. Entre as áreas que aparecem com mais clareza nas referências estudadas estão a otimização de funções, o aprendizado de máquina, no caso das redes neurais, e a engenharia, já que Karaboga e Basturk (2008) destacam que o algoritmo pode ser aplicado a problemas de engenharia de alta dimensão.

5 EXEMPLO PRÁTICO

Para o exemplo prático o ABC foi implementado em Python, na forma descrita na seção 3, com uma variável alterada por visita, seleção gulosa entre o ponto novo e a própria fonte, roleta para as observadoras e uma exploradora por ciclo. Primeiro foi rodado o problema didático de minimizar $f(x) = x^2$ com x entre -10 e 10, usando 6 fontes, limite 6 e 100 ciclos, com semente fixa (2005), que é o mesmo exemplo usado na animação e nos slides da apresentação.

Depois, para ver o efeito dos parâmetros, o algoritmo foi aplicado à função Rastrigin com duas variáveis e domínio de -5,12 a 5,12, cujo mínimo global vale 0 em (0, 0). Cada configuração foi executada 30 vezes, com sementes de 1 a 30, por 40 ciclos, e foram medidos a mediana do melhor valor encontrado, a porcentagem de execuções que chegaram a f menor que 10^{-3} (chamada aqui de taxa de acerto) e o número médio de vezes que a exploradora

trocou uma fonte. No primeiro grupo o limite variou entre 2, 10, 40 e 200 com 20 fontes, e no segundo o número de fontes variou entre 5, 10, 20 e 40 com limite 40. A função Sphere com 20 fontes e limite 40 serviu de referência.

Para mostrar o que acontece dentro de um ciclo, antes dos testes maiores, foi usado um roteiro fixo com três fontes, A em x = 7, B em x = -5 e C em x = 2, e limite 3, que é o mesmo da animação. As empregadas visitam na ordem A, B, C e depois as chances da roleta são calculadas pela qualidade de cada fonte, ficando em cerca de 33% para A, 3% para B e 65% para C, o que faz sentido porque C está bem mais perto do zero. A roleta sorteia então C, A e B para as observadoras, e a Tabela 1 resume os seis passos.

Tabela 1 - Um ciclo do ABC em f(x) = x², com três fontes e limite 3

Passo	Abelha (fonte, vizinha)	Ponto novo	Resultado
1	Empregada (A, B)	1,0	melhor que 7: A vai para 1,0
2	Empregada (B, C)	-7,8	pior que -5: B fica, falhas = 1
3	Empregada (C, B)	-0,1	melhor que 2: C vai para -0,1
4	Observadora (C, A)	-0,65	pior que -0,1: C fica, falhas = 1
5	Observadora (A, C)	0,12	melhor que 1,0: A vai para 0,12
6	Observadora (B, A)	-0,392	melhor que -5: B vai para -0,392, falhas = 0

Fonte: elaborado pelo autor.

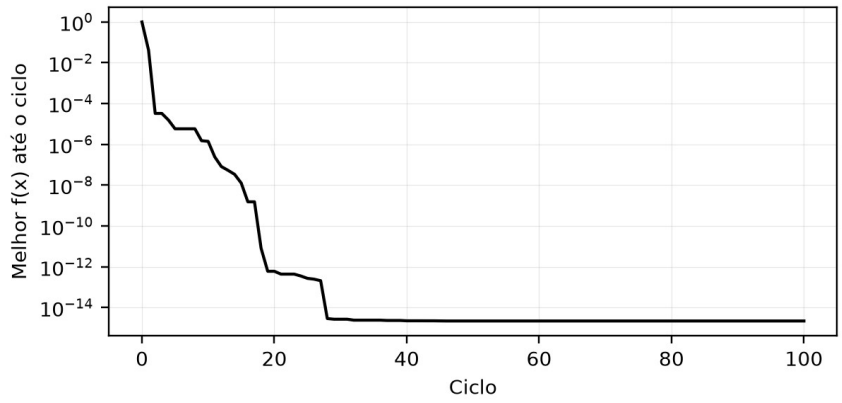
Ao final do ciclo o melhor ponto é o de C (f = 0,01), e nenhuma fonte passou do limite 3, então a exploradora não age. Quem teve duas visitas melhores seguidas foi A, que saiu de 7 e chegou a 0,12 em um ciclo só, e o que chama atenção no passo 2 é que a vizinha serve só para definir o tamanho do passo: B foi puxada para -7,8, mais longe do zero, e por isso continuou onde estava. Na animação existe ainda uma cena hipotética onde B chega a 4 falhas, murcha, e a exploradora sorteia o ponto -7,2, que é aceito mesmo sendo pior que a fonte anterior.

6 RESULTADOS E DISCUSSÃO

No problema x², o melhor valor de f caiu de 0,985 no início para $3{,}3 \times 10^{-5}$ já no segundo ciclo e para $2{,}2 \times 10^{-15}$ por volta do ciclo 50, como mostra a Figura 1, e depois disso o valor ficou parado, pois o melhor ponto está praticamente em x = 0 e não há mais o que

melhorar. Nesse mesmo período a exploradora agiu 32 vezes nos 100 ciclos, o que parece estranho mas tem explicação: com todas as fontes já perto do zero, as visitas deixam de dar um ponto melhor, as falhas se acumulam e o limite 6 é ultrapassado, nisso a fonte é trocada por um ponto sorteado que, em geral, é bem pior, sem prejudicar o melhor valor guardado.

Figura 1 - Melhor valor de f(x) = x² por ciclo (6 fontes, limite 6, semente 2005)



Fonte: elaborado pelo autor.

Na Rastrigin, o efeito do limite foi o mais claro dos testes (Tabela 2). Com limite 2 a exploradora trocou uma fonte em praticamente todos os 40 ciclos (39,8 em média) e só 10% das execuções chegaram ao mínimo global, pois as fontes são abandonadas antes de serem refinadas, e com limite 10 a taxa subiu para 40%. Com limite 40 a exploradora quase não agiu (0,3 vezes) e a taxa foi de 100%, e com limite 200 ela não agiu nenhuma vez e a taxa foi de 97%, ou seja, uma das 30 execuções não chegou lá em 40 ciclos, o que sugere que um limite alto demais prende a colônia numa região sem a ajuda de ninguém para sair.

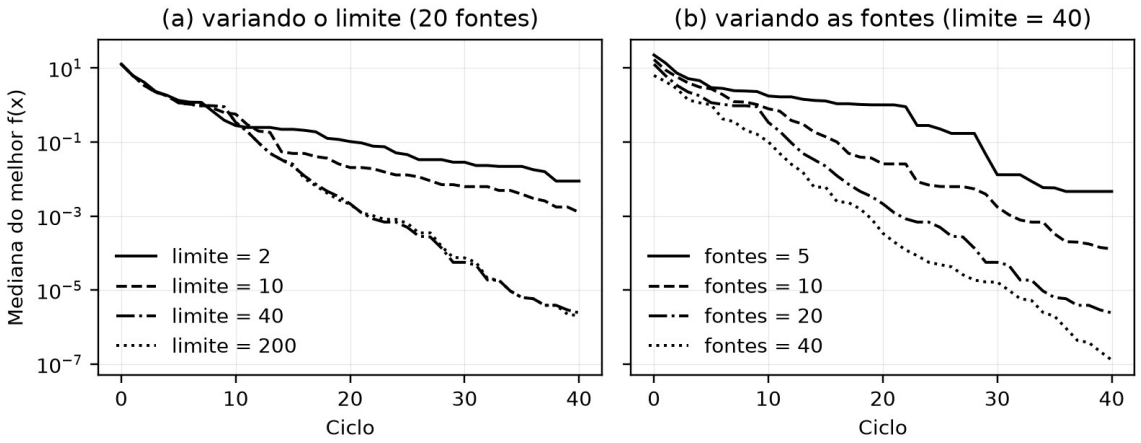
Tabela 2 - Resultados na função Rastrigin com duas variáveis (30 execuções, 40 ciclos)

Configuração	**Mediana do melhor f**	**Taxa de acerto**	**Trocas pela exploradora**
20 fontes, limite 2	$8{,}8 \times 10^{-3}$	10%	39,8
20 fontes, limite 10	$1{,}3 \times 10^{-3}$	40%	26,4
20 fontes, limite 40	$2{,}4 \times 10^{-6}$	100%	0,3
20 fontes, limite 200	$2{,}0 \times 10^{-6}$	97%	0,0
5 fontes, limite 40	$4{,}6 \times 10^{-3}$	40%	0,1
10 fontes, limite 40	$1{,}3 \times 10^{-4}$	60%	0,4
20 fontes, limite 40	$2{,}4 \times 10^{-6}$	100%	0,3
40 fontes, limite 40	$1{,}3 \times 10^{-7}$	100%	0,6

Fonte: elaborado pelo autor.

Quanto ao número de fontes, com 5 a taxa de acerto foi de 40% e a mediana do melhor f foi de $4{,}6 \times 10^{-3}$, com 10 foi de 60%, e com 20 e 40 fontes todas as execuções chegaram ao mínimo global, com a mediana melhorando de $2{,}4 \times 10^{-6}$ para $1{,}3 \times 10^{-7}$, conforme a Figura 2. Como em cada ciclo o algoritmo avalia a função da ordem de duas vezes o número de fontes (uma visita por empregada e uma por observadora), 40 fontes gastam o dobro de avaliações de 20 para um ganho que, na taxa de acerto, já não aparece. Na Sphere, que tem um vale só, a mediana final foi de $3{,}0 \times 10^{-12}$ e todas as execuções acertaram, bem mais fácil que a Rastrigin com os mesmos parâmetros.

Figura 2 - Mediana do melhor f(x) por ciclo na Rastrigin, variando (a) o limite e (b) o número de fontes



Fonte: elaborado pelo autor.

Vale dizer que esses resultados valem para uma função de teste com duas variáveis e 40 ciclos, e que em dimensões maiores, onde a Rastrigin tem muito mais vales, os valores de limite e de fontes que funcionaram aqui provavelmente não são os melhores. Também não foi testada nenhuma outra metaheurística no mesmo código, então a comparação feita por Karaboga e Basturk (2007; 2008) não foi reproduzida, apenas o comportamento do ABC consigo mesmo.

Todos os números aqui saem do código em Python do trabalho, onde cada execução usa uma semente de 1 a 30, e rodando de novo com as mesmas sementes o resultado é o mesmo. A taxa de acerto de 10^{-3} foi uma escolha minha para separar quem achou o vale central de quem ficou num vale falso, e com outro valor de corte os percentuais mudariam um pouco, ainda que a ordem entre as configurações dificilmente mude, já que as medianas contam a mesma história.

Do lado das vantagens, o ABC é simples de implementar e tem poucos parâmetros (KARABOGA; BASTURK, 2008), e a própria implementação deste trabalho tem algumas dezenas de linhas. Do lado das limitações, ele não garante o mínimo global, como toda

metaheurística, e os testes mostraram que dois desses poucos parâmetros, o limite e o número de fontes, mudam bastante o resultado.

7 CONCLUSÃO

O ABC resolve o problema de procurar bons valores para uma função difícil sem precisar de derivadas e sem testar todas as possibilidades, usando uma colônia dividida em três papéis, onde as empregadas refinam, as observadoras concentram a busca nas fontes boas e a exploradora abandona as que não melhoram. Nos testes realizados na Rastrigin com duas variáveis, limite entre 40 e 200 e 20 ou mais fontes foram suficientes para encontrar o mínimo global em quase todas as execuções, enquanto limite 2 e 5 fontes deixaram a taxa de acerto em 10% e 40%.

REFERÊNCIAS

KARABOGA, D. *An idea based on honey bee swarm for numerical optimization*. Kayseri: Erciyes University, Engineering Faculty, Computer Engineering Department, 2005. (Technical Report TR06).

KARABOGA, D.; AKAY, B.; ÖZTÜRK, C. Artificial bee colony (ABC) optimization algorithm for training feed-forward neural networks. In: TORRA, V.; NARUKAWA, Y.; YOSHIDA, Y. (ed.). *Modeling decisions for artificial intelligence*: MDAI 2007. Berlin: Springer, 2007. p. 318-329. (Lecture Notes in Computer Science, v. 4617).

KARABOGA, D.; BASTURK, B. A powerful and efficient algorithm for numerical function optimization: artificial bee colony (ABC) algorithm. *Journal of Global Optimization*, v. 39, n. 3, p. 459-471, 2007. DOI: 10.1007/s10898-007-9149-x.

KARABOGA, D.; BASTURK, B. On the performance of artificial bee colony (ABC) algorithm. *Applied Soft Computing*, v. 8, n. 1, p. 687-697, 2008. DOI: 10.1016/j.asoc.2007.05.007.

KARABOGA, D.; GORKEMLI, B.; OZTURK, C.; KARABOGA, N. A comprehensive survey: artificial bee colony (ABC) algorithm and applications. *Artificial Intelligence Review*, v. 42, n. 1, p. 21-57, 2014. DOI: 10.1007/s10462-012-9328-0.